



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS EN LA COMPUTACIÓN Y DISEÑO DIGITAL

Integrantes:

Cando Moreno Robinson Rodrigo

Cedeño Briones Mecias Saul

Mendoza Triana Jaider Joel

Asignatura:

Administración De Bases De Datos

Docente:

Ing. CORDERO BAZURTO JOSE STEVEN

Curso/Paralelo:

V Semestre-Software "B"



1. Plan de Respaldos y Recuperación

Este documento describe la estrategia de respaldo (backup) y recuperación (restore) implementada para la base de datos soportenet_isp, incluyendo el tipo de respaldo elegido, su justificación, la frecuencia recomendada, el procedimiento de restauración y las medidas adoptadas para minimizar el riesgo de pérdida de información. Todas las pruebas descritas fueron ejecutadas sobre la base con volumen real de datos.

1.1 Tipo de respaldo

Se utiliza un respaldo lógico completo mediante la herramienta `pg_dump`, en formato custom (-Fc), el cual queda comprimido y permite una restauración en paralelo. Se optó por un respaldo lógico en lugar de uno físico basado en WAL por las siguientes razones:

- Es portable entre distintas versiones de PostgreSQL.
- No requiere configuración adicional en el servidor (como archivado continuo de WAL).
- Resulta adecuado para el volumen de datos actual del proyecto.

```
pg_dump -h localhost -U postgres -d soportenet_isp -Fc -f
respaldo_soportenet.dump
```

Prueba real: con la base cargada (~3.78 millones de registros), el respaldo tardó 26.8 segundos y generó un archivo de 34.7 MB.

1.2 Frecuencia recomendada

- Ejecución diaria, en horario de bajo tráfico (mañana), automatizada mediante una tarea programada `cron` en Linux o Task Scheduler en Windows que ejecute el comando de respaldo incluyendo la fecha en el nombre del archivo.
- Política de retención: mantener los últimos 7 respaldos diarios más 1 respaldo semanal de las últimas 4 semanas. Esto es suficiente para recuperar información ante un error detectado tardíamente, sin acumular espacio de almacenamiento de forma indefinida.

1.3 Procedimiento de restauración

Paso 1 Crear la base de datos destino (vacía):

```
createdb -h localhost -U postgres soportenet_restaurada
```

Paso 2 Restaurar el respaldo (con paralelismo para acelerar en bases grandes):

```
pg_restore -h localhost -U postgres -d soportenet_restaurada \
--no-owner -j 4 respaldo_soportenet.dump
```

Verificación de integridad

Se restauró el respaldo de 3.78 millones de registros en una base nueva en 41 segundos, y se verificó que el conteo de filas coincidiera exactamente con el original en las tablas principales:

Tabla	Original	Restaurada
solicitud	1,000,000	1,000,000
usuario	103,070	103,070

Restauración confirmada 100% íntegra.

1.4 Medidas ante posible pérdida de información

- No guardar los respaldos en el mismo disco o máquina que la base de datos: si el disco falla, se pierde la base y el respaldo al mismo tiempo. Como mínimo, copiarlos a almacenamiento externo o a la nube (Google Drive, S3, etc.).
- Verificar el respaldo periódicamente restaurándolo en una base de prueba, tal como se hizo en este documento. Un respaldo que nunca se probó restaurar no es un respaldo confiable, es solo un archivo del que se espera que funcione.
- Las tablas de negocio no permiten borrado físico de usuarios (solo cambio de estado_cuenta), lo que reduce el riesgo de pérdida de información por error humano en operación normal.
- El bloqueo optimista (solicitud.version) protege contra pérdida de cambios por condiciones de carrera entre dos actualizaciones simultáneas.
- Si el volumen de datos creciera considerablemente, la siguiente palanca sería un respaldo incremental basado en WAL (pg_basebackup + archivado continuo), que permite recuperación a un punto exacto en el tiempo. No fue necesario para el volumen actual del proyecto.

2. Optimización de Consultas

El análisis de desempeño se realizó sobre la base ya cargada con más de 3.7 millones de registros (1,000,000 en la tabla solicitud), de modo que las mediciones reflejen el comportamiento real del sistema a escala, y no el de una tabla vacía donde cualquier plan de ejecución resulta "rápido" sin importar si existen o no los índices adecuados.

2.1 Caso 1 "Mis tareas" del técnico

Consulta real utilizada por `SolicitudRepository.findMisTareas`, la cual trae las solicitudes asignadas directamente a un técnico o a un grupo del que dicho técnico es miembro.

Antes

```
SELECT s.* FROM solicitud s
JOIN asignacion_solicitud a ON a.id_solicitud = s.id_solicitud AND
a.vigente = true
LEFT JOIN tecnico_grupo tg ON tg.id_grupo = a.id_grupo AND
tg.id_usuario = 100011
LEFT JOIN prioridad p ON p.id_prioridad = s.id_prioridad
WHERE a.id_tecnico = 100011 OR tg.id_usuario = 100011
ORDER BY COALESCE(p.orden, 0) DESC, s.fecha_creacion ASC
LIMIT 20;
```

EXPLAIN ANALYZE mostró que, a pesar de que ya existe el índice `idx_asignacion_tecnico_vigente`, la consulta ejecuta un Parallel Seq Scan sobre las 300,000 filas de `asignacion_solicitud` el índice no se utiliza. La causa es que la condición `WHERE a.id_tecnico = X OR tg.id_usuario = X` combina dos tablas distintas mediante un OR, y PostgreSQL no puede aplicar el índice de una tabla cuando la condición depende también del resultado del JOIN con la otra.

Resultado: 389.674 ms de ejecución.

Después (reescrita como UNION)

```
SELECT s.*, p.orden AS p_orden FROM (
    SELECT a.id_solicitud FROM asignacion_solicitud a
    WHERE a.id_tecnico = 100011 AND a.vigente = true
    UNION
    SELECT a.id_solicitud FROM asignacion_solicitud a
    JOIN tecnico_grupo tg ON tg.id_grupo = a.id_grupo AND tg.id_usuario
= 100011
    WHERE a.vigente = true
) mis_asignaciones
JOIN solicitud s ON s.id_solicitud = mis_asignaciones.id_solicitud
LEFT JOIN prioridad p ON p.id_prioridad = s.id_prioridad
ORDER BY COALESCE(p.orden, 0) DESC, s.fecha_creacion ASC
LIMIT 20;
```

Separar el OR en dos SELECT unidos por UNION permite que cada rama utilice su propio índice: la primera usa idx_asignacion_tecnico_vigente directamente, y la segunda usa idx_asignacion_grupo_vigente.

Resultado: 14.027 ms de ejecución aproximadamente 27.8 veces más rápido.

2.2 Caso 2 Conteo de solicitudes por estado (dashboard del administrador)

Consulta real utilizada por el panel de administrador para las tarjetas de métricas (endpoint /api/solicitudes?estado=Cerrada&size=1 → totalElements).

```
SELECT count(*) FROM solicitud s
JOIN estado e ON e.id_estado = s.id_estado
WHERE e.nombre_estado = 'Cerrada';
```

Se probó quitando y volviendo a poner el índice idx_solicitud_estado, en una copia de la base restaurada del respaldo, para no afectar la base real:

Plan de ejecución	Escenario	Tiempo
Index Only Scan using idx_solicitud_estado	Con índice	46 – 52 ms
Parallel Seq Scan sobre 1,000,000 filas	Sin índice	509 ms

Aproximadamente 11 veces más rápido con el índice. Este índice ya existía en el script base (idx_solicitud_estado); la prueba confirma que efectivamente está cumpliendo su función.

2.3 Conclusión

- Tener un índice no es suficiente por sí solo: en el Caso 1, el índice correcto ya existía, pero la forma en que estaba escrita la consulta impedía su uso fue necesario reescribir la consulta, no solo indexar.
- En el Caso 2, el índice existente sí se estaba aprovechando correctamente, y la prueba lo confirma con evidencia medible de antes y después.
- Ambas pruebas se ejecutaron sobre la base con el volumen completo (1M+ filas en solicitud), para que los tiempos reflejen un caso de uso real y no una tabla vacía donde cualquier plan resulta "rápido" sin importar el índice.